

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

ОТЧЁТ ПО УЧЕБНОЙ ПРАКТИКЕ

Тема: НОВЫЕ ЯЗЫКИ ПРОГРАММИРОВАНИЯ

Студент гр. 4382

Вязников Н.К.

Студент гр. 4382

Королев С.Е.

Студент гр. 4342

Михайлов М.А.

Руководитель

Шестопалов Р.П.

Санкт-Петербург

2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студент: Вязников Н.К. Группа: 4382

Студент: Королев С.Е. Группа: 4382

Студент: Михайлов М.А. Группа: 4342

Тема практики: Новые языки программирования

Задание на практику:

Командная поэтапная разработка программы визуализации алгоритмов на Java с графическим интерфейсом.

Алгоритм: Топологическая сортировка.

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 16.07.2026

Дата защиты отчета: 17.07.2026

Студент

Вязников Н.К.

Студент

Королев С.Е.

Студент

Михайлов М.А.

Руководитель

Шестопалов Р.П.

АННОТАЦИЯ

В рамках учебной практики разработано приложение на Java/Swing для визуализации топологической сортировки ориентированных ациклических графов. Реализованы алгоритмы Кана (очередь) и DFS (стек). Пользователь может создавать граф интерактивно или загружать из файла, наблюдать пошаговое выполнение с цветовой индикацией состояний вершин, возвращаться на шаг и сбрасывать процесс. Программа корректно обрабатывает циклические графы. Проект позволил закрепить навыки объектно-ориентированного программирования и разработки графических интерфейсов.

SUMMARY

This paper describes a Java/Swing application for visualizing topological sorting of directed acyclic graphs. It implements Kahn's algorithm (queue) and DFS-based sorting (stack). Users can build graphs interactively or load them from files, step through execution with color-coded vertex states, roll back steps, and reset the process. The program handles cyclic graphs gracefully. The internship strengthened skills in OOP and GUI development.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 ПОСТАНОВКА ЗАДАЧИ	7
2 РЕЗУЛЬТАТЫ РАБОТЫ.....	8
2.1. Вводное задание.....	8
2.2. Составление спецификации программы и плана разработки	8
2.2.1. Исходные требования к программе	8
2.2.1.1. Требования к вводу исходных данных.....	8
2.2.1.2. Требования к визуализации.....	8
2.2.1.3. Требования к функционалу приложения	9
2.2.2. План разработки и распределение ролей в бригаде.....	9
2.3. Командная итеративная разработка приложения	10
2.3.1. Структуры данных.....	10
2.3.2. Основные методы	11
3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ	14
4 ОТЗЫВ РУКОВОДИТЕЛЯ	15
ЗАКЛЮЧЕНИЕ	16
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	17
ПРИЛОЖЕНИЕ А	18

ВВЕДЕНИЕ

В рамках учебной практики перед бригадой была поставлена цель разработать программный продукт для визуализации алгоритмов топологической сортировки ориентированных ациклических графов. Приложение позволяет в интерактивном режиме наблюдать пошаговое выполнение алгоритмов, что делает его полезным инструментом для изучения теории графов как студентами, так и преподавателями.

Предметной областью практики является разработка десктопного приложения на языке Java с использованием библиотеки Swing. Визуализация работы алгоритмов в реальном времени с цветовой индикацией состояний вершин обеспечивает наглядное понимание логики сортировки и позволяет сравнивать поведение двух различных подходов: на основе очереди (алгоритм Кана) и на основе стека (DFS-алгоритм).

Целью практики было создание интерактивного графического приложения, предоставляющего пользователю возможность конструировать граф с помощью мыши или загружать его из текстового файла, выбирать метод сортировки и пошагово отслеживать все изменения структуры данных и состояния вершин вплоть до получения итогового топологического порядка.

Для достижения поставленной цели были сформулированы следующие задачи практики:

- Изучение библиотеки Swing и принципов построения графических интерфейсов на Java, освоение работы с событиями мыши и клавиатуры для реализации интерактивного редактирования графа.
- Разработка архитектуры приложения, включающей модели данных (вершины и рёбра), компоненты визуализации и алгоритмическое ядро с поддержкой пошагового выполнения.
- Реализация двух алгоритмов топологической сортировки (на основе очереди и на основе стека) с возможностью возврата на предыдущий шаг, сброса и автоматического воспроизведения.

- Создание редактора графа с интерактивным управлением, поддержкой масштабирования и перемещения холста, а также механизмов загрузки и сохранения графа в различных текстовых форматах.
- Разработка легенды и информационных панелей для отображения текущего состояния алгоритма и результата сортировки.
- Проведение тестирования корректности работы алгоритмов на графах различной структуры, включая циклические, и отладка пользовательского интерфейса.
- Оформление отчётной документации, описывающей процесс разработки, архитектуру и функциональные возможности созданного продукта.

1 ПОСТАНОВКА ЗАДАЧИ

В результате была разработана программа, полностью соответствующая утверждённой спецификации. Приложение имеет графический интерфейс на основе библиотеки Swing, позволяет взаимодействовать с графом исключительно с помощью мыши, предоставляет пользователю возможность задавать граф как интерактивно на холсте, так и загружая его из текстового файла в одном из трёх поддерживаемых форматов. Работа алгоритмов топологической сортировки демонстрируется пошагово и полностью контролируется пользователем через панель управления.

2 РЕЗУЛЬТАТЫ РАБОТЫ

2.1. Вводное задание

На начальном этапе практики был пройден курс на степике[1] было проведено изучение библиотеки Swing[2] и принципов построения графических интерфейсов на Java. В ходе самостоятельной подготовки были освоены темы: работа с контейнерами и менеджерами компоновки, обработка событий мыши и клавиатуры, механизмы перерисовки компонентов, использование аффинных преобразований для масштабирования и панорамирования, а также сериализация объектов для сохранения состояния приложения.

2.2. Составление спецификации программы и плана разработки

В данном разделе приводятся исходная спецификация, план разработки и распределение ролей в бригаде, согласованные с руководителем, а также уточнения требований, принятые после сдачи промежуточных версий программы.

2.2.1. Исходные требования к программе

2.2.1.1. Требования к вводу исходных данных

Входные данные могут быть заданы пользователем интерактивно с помощью мыши на холсте или загружены из текстового файла. При загрузке из файла поддерживаются три формата: список рёбер, матрица смежности и список смежности. Первая строка файла всегда содержит количество вершин. В формате списка рёбер каждая последующая строка содержит два целых числа - начальную и конечную вершины ребра. Матрица смежности задаётся квадратной таблицей, где на пересечении строки i и столбца j указывается 1 при наличии ребра и 0 в противном случае. Список смежности описывается построчно: в каждой строке первое число - вершина, остальные - её соседи. Поскольку алгоритм работает с ориентированным графом, порядок указания вершин в ребре имеет значение: первое значение указывает на вершину, из которой выходит ребро, второе - на вершину, в которую оно входит.

2.2.1.2. Требования к визуализации

Интерфейс разделён на три области: центральная часть - холст для отображения графа, где пользователь создаёт вершины и рёбра. Вершины отображаются в виде кругов с цветовой индикацией состояния: белый - не обработана, серый - в процессе обработки, чёрный - обработана. Рёбра отображаются в виде направленных линий со стрелками. Правая часть - легенда с пояснением цветовой схемы. Нижняя часть - панель управления, содержащая кнопки для выполнения шагов вперёд и назад, остановки, а также поле для вывода результата сортировки. В верхней части расположена панель инструментов с кнопками загрузки и сохранения графа, а также выпадающим списком для выбора алгоритма сортировки.

2.2.1.3. Требования к функционалу приложения

Программа позволяет: создавать и редактировать ориентированный граф; выбирать алгоритм сортировки из двух доступных (на основе очереди и на основе стека); пошагово выполнять алгоритм с визуализацией изменений состояний вершин; возвращаться на предыдущий шаг; сбрасывать выполнение в начальное состояние; автоматически воспроизводить алгоритм с заданной задержкой; выявлять циклические графы и выдавать соответствующее сообщение; отображать итоговый порядок отсортированных вершин. Все функции доступны через графический интерфейс и не требуют использования командной строки.

2.2.2. План разработки и распределение ролей в бригаде

Разработка велась итеративно с разделением обязанностей между членами бригады.

Этап 1, прототип (день 1–2). Реализация базовых классов модели (Node, Edge), разработка компонента JGraph для отрисовки графа, реализация обработки событий мыши для добавления вершин и рёбер.

Этап 2, альфа (день 3–4). Реализация абстрактного класса TopologicalSort и его наследников - TopologicalSortQueue (алгоритм Кана) и TopologicalSortStack (DFS-алгоритм). Внедрение пошагового режима с цветовой индикацией.

Этап 3, бета (день 5–6). Создание полноценного пользовательского интерфейса: панель управления (JPlayer), панель инструментов (JGraphToolBar), легенда (JLegend), главное окно (JRootFrame). Реализация загрузки и сохранения графа через JLoadGraph.

Этап 4, релиз (день 6–7). Проведение тестирования, устранение дефектов, доработка интерфейса на основе замечаний, оформление документации.

Роли были распределены следующим образом:

Разработчик 1 – Вязников Никита Константинович (UI/интеграция) – реализация главного окна, панелей инструментов и управления, связывание интерфейса с алгоритмами, обработка событий кнопок, вывод результатов и сообщений об ошибках.

Разработчик 2 – Михайлов Мирон Артемьевич (модель и визуализация) – реализация классов Node, Edge, JGraph (отрисовка и масштабирование), обработка событий мыши на холсте, загрузка и сохранение графа в текстовый файл.

Разработчик 3 – Королев Семен Ефремович (алгоритмы) – реализация абстрактного класса TopologicalSort и двух его наследников, логика пошагового выполнения и отката, управление состояниями вершин, обнаружение циклов, модульное тестирование алгоритмов.

2.3. Командная итеративная разработка приложения

Разработка велась в соответствии с утверждённым планом. В процессе работы промежуточные версии программы регулярно демонстрировались руководителю, что позволяло своевременно корректировать требования и дорабатывать функционал. Благодаря чёткому разделению обязанностей и регулярной синхронизации результатов между разработчиками интеграция отдельных модулей проходила без значительных затруднений. Использование системы контроля версий Git позволило эффективно управлять изменениями и фиксировать промежуточные состояния проекта.

2.3.1. Структуры данных

Класс Node описывает вершину графа и содержит координаты на холсте, цветовое состояние и флаг посещения (для DFS-алгоритма).

Класс Edge описывает ориентированное ребро и содержит идентификаторы начальной и конечной вершин.

Класс JGraph является компонентом визуализации, содержит списки вершин и рёбер, отвечает за их отрисовку на холсте с учётом масштабирования и сдвига.

Абстрактный класс TopologicalSort определяет единый интерфейс для алгоритмов сортировки: инициализация, шаг вперёд, шаг назад, сброс, проверка начального и конечного состояний.

Класс TopologicalSortQueue реализует алгоритм Кана с использованием очереди Queue<Integer>. Хранит текущие значения входящих степеней вершин и очередь готовых к обработке вершин.

Класс TopologicalSortStack реализует алгоритм на основе поиска в глубину с использованием стека. Дополнительно содержит структуры для хранения цветовых состояний (белый/серый/чёрный), булевый массив посещённых вершин и стек для накопления результата.

2.3.2. Основные методы

Класс JGraph:

paintComponent(Graphics g) - основная отрисовка графа: сетка, рёбра со стрелками, вершины с цветовой заливкой. Использует аффинные преобразования для масштабирования.

Обработчики событий мыши - добавление вершин по щелчку, создание рёбер между выбранными вершинами, перемещение вершин перетаскиванием, удаление через контекстное меню.

setGraph(ArrayList<Node>, ArrayList<Edge>) - загрузка нового графа в компонент с последующей перерисовкой.

Класс TopologicalSortQueue:

`init()` - инициализация алгоритма: вычисление входящих степеней всех вершин, добавление вершин с нулевой степенью в очередь, окрашивание их в серый цвет.

`stepForward()` - извлечение вершины из очереди, добавление её в результат, окрашивание в чёрный цвет, уменьшение входящих степеней соседей, добавление в очередь вершин с обнулившейся степенью.

`stepBack()` - откат последнего шага: удаление вершины из результата, восстановление входящих степеней соседей, возврат вершины в очередь.

`isEnd()` / `isBegin()` - проверка завершённости алгоритма или начального состояния.

Класс `TopologicalSortStack`:

`init()` - сброс всех вершин в состояние белого цвета.

`stepForward()` - запуск DFS-обхода от следующей необработанной вершины. При входе в вершину - окрашивание в серый цвет и помещение в стек. При выходе - окрашивание в чёрный и добавление в результирующий стек.

`stepBack()` - откат последнего действия: либо возврат вершины из результирующего стека в состояние серого, либо возврат к белой вершине.

`isEnd()` / `isBegin()` - проверка завершённости или начального состояния.

Класс `JLoadGraph`:

Загрузка графа из файла с определением типа формата (список рёбер, матрица смежности, список смежности). При успешной загрузке выполняется автоматическая расстановка вершин с помощью силового алгоритма для равномерного распределения по холсту.

Валидация входных данных с выводом информативных сообщений об ошибках.

Класс `JGraphPopupMenu`:

`saveState(File)` - сериализация текущего состояния программы (граф, выбранный алгоритм, состояние алгоритма) в бинарный файл.

readState(File) - десериализация состояния из файла с восстановлением графа и алгоритма в соответствующем шаге выполнения.

Тестирование разработанного приложения проводилось вручную на протяжении всего цикла разработки. Проверялись как отдельные модули (вычисления входящих степеней, логика шагов алгоритмов), так и работа интерфейса в целом - создание и редактирование графа, загрузка из файлов различных форматов, пошаговое выполнение, возврат на предыдущие шаги, обнаружение циклических графов. Особое внимание уделялось проверке корректности работы алгоритмов на графах с различной структурой: связных и несвязных, с одной и несколькими вершинами без рёбер. Все тесты подтвердили правильность реализации: программа стабильно выполняет сортировку на ациклических графах, выдаёт корректный порядок вершин и однозначно идентифицирует наличие циклов с выводом соответствующего сообщения. Интерфейс остаётся отзывчивым при всех операциях, ошибок ввода-вывода и непредвиденных ситуаций зафиксировано не было.

3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

В ходе работы был использован язык программирования Java. Весь алгоритмический модуль, а также графический интерфейс и логика взаимодействия компонентов реализованы на этом языке без применения внешних фреймворков.

Графический интерфейс: библиотека Swing[2] (классы JFrame, JPanel, JToolBar, JPopupMenu, JComboBox, JFileChooser). Для отрисовки графа на холсте использовались Graphics2D, аффинные преобразования (AffineTransform) для масштабирования и панорамирования, а также механизмы антиалиасинга для сглаживания изображений. Обработка событий мыши и клавиатуры реализована через встроенные слушатели (MouseListener, MouseMotionListener, MouseWheelListener).

Сериализация данных: стандартная сериализация Java (ObjectOutputStream / ObjectInputStream) для сохранения и загрузки состояния программы, включая граф, выбранный алгоритм и прогресс его выполнения.

Работа с файлами: для загрузки и сохранения графов в текстовых форматах использованы FileInputStream и FileOutputStream с ручным парсингом и валидацией входных данных (список рёбер, матрица смежности, список смежности). Выбор файлов организован через диалоговое окно JFileChooser с фильтром расширений.

Структура проекта: модульная организация исходного кода с разделением на пакеты src.main (модель данных, визуализация и интерфейс) и src.main.Algorithms (реализации алгоритмов сортировки). Для управления версиями использовалась система Git.

4 ОТЗЫВ РУКОВОДИТЕЛЯ

По результатам работы учебная практика оценена на Отлично. (рис. 1)

Отзыв
о прохождении учебной практики

Королев Семен Ефремович, студент группы 4382, второго курса бакалавриата Санкт-Петербургского электротехнического университета "ЛЭТИ" им В.И. Ульянова, проходил учебную практику по теме "Новые языки программирования" с 06.07.2026 по 18.07.2026.

В течение практики обучающийся Королев Семен Ефремович выполнял задание по разработке модуля алгоритмов топологической сортировки для приложения визуализации ориентированных ациклических графов. В его обязанности входило: проектирование и реализация двух алгоритмов сортировки с поддержкой пошагового выполнения, отката, сброса состояния, а также интеграция этих алгоритмов в общую архитектуру приложения на Java с графическим интерфейсом Swing.

В результате практики обучающийся полностью реализовал требуемый функционал: разработал абстрактный базовый класс, две его реализации – на очереди и стеке, обеспечил управление состояниями вершин и корректное обнаружение циклов в графе. Все алгоритмы прошли модульное тестирование на различных графах и успешно интегрированы в основное приложение, что подтверждено работоспособностью программы в целом. Получаемую работу обучающийся Королев Семен Ефремович выполнял самостоятельно и ответственно. Следует отметить, что основные этапы работы были завершены в установленные сроки, а часть задач – даже с некоторым опережением. Студент регулярно информировал о промежуточных результатах, оперативно реагировал на замечания, а также предложил улучшение логики отката для алгоритма на основе стека, что положительно сказалось на удобстве визуализации.

По результатам работы Королев Семен Ефремович учебную практику оцениваю на Отлично.

Руководитель практики
Ассистент каф.МОЭВМ 15.07.2026

 / Шестопалов Р.П.

Рисунок 1 - Отзыв руководителя

ЗАКЛЮЧЕНИЕ

В ходе учебной практики была выполнена командная разработка программного продукта[3] - визуализатора алгоритмов топологической сортировки на языке Java с использованием библиотеки Swing[2]. Полностью решены все поставленные задачи: изучены основы Swing[2] и принципы построения графических интерфейсов, разработана архитектура приложения с разделением на модель данных, компоненты визуализации и алгоритмическое ядро, реализованы два алгоритма сортировки (на основе очереди и на основе стека) с поддержкой пошагового выполнения, отката и сброса, создан интерактивный редактор графа с управлением через мышь, реализованы механизмы загрузки и сохранения графа в нескольких текстовых форматах, проведено ручное тестирование на различных типах графов (ациклические, циклические, несвязные, с различным количеством вершин и рёбер), а также проверена корректность работы интерфейса во всех режимах - ошибок и непредвиденного поведения не выявлено.

Разработанное приложение полностью соответствует утверждённой спецификации и может использоваться в учебном процессе для наглядной демонстрации работы алгоритмов топологической сортировки, а также для сравнения двух подходов к решению задачи. Полученные навыки объектно-ориентированного программирования на Java, проектирования графических интерфейсов и командной разработки с использованием системы контроля версий будут полезны в дальнейшей профессиональной деятельности.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Онлайн-курс Java [Электронный ресурс]. URL: <https://stepik.org/course/Java-Базовый-курс-187/syllabus> (дата обращения: 05.07.2026).
2. Java Platform, Standard Edition API Specification: javax.swing [Электронный ресурс]. URL: <https://docs.oracle.com/javase/8/docs/api/javax/swing/package-summary.html> (дата обращения: 09.07.2026).
3. Github [Электронный ресурс]. URL: <https://github.com/Gwynbleidd2516/Summer-practice> (дата обращения: 06.07.2026)

ПРИЛОЖЕНИЕ А

Представляется финальный вид пользовательского интерфейса приложения (рис. 2).

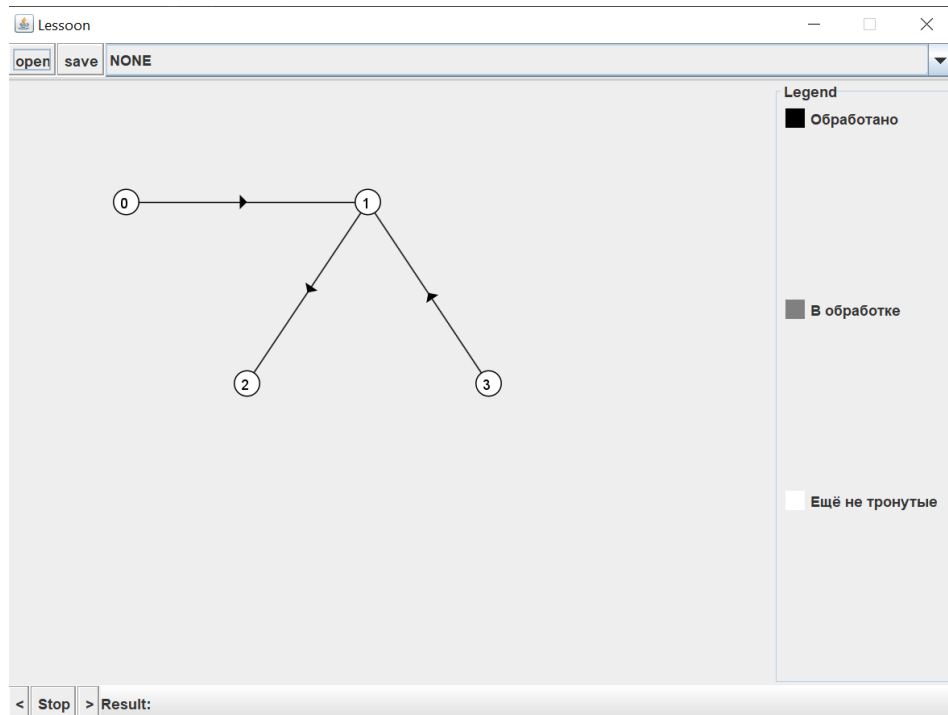


Рисунок 2 – Интерфейс программы